

Welcome to Big Data!!!

Today we are going to analyze Rainfall data from central Park to get you started

We are going to do this all in a Jupyter notebook which was formerly called iPython. This will take some getting used to. You actually do your programming in a window in a web browser or in a program. But you are not online. You are running off of your own computer. But we will teach you all you need to do. Make sure to type everything you see correctly. All syntax matters including capital letters and punctuation.

Ok, Lets get you started. But make sure you read the syllabus and get a binder to keep the handouts. You need to do the code.org also!

We are going to use [Anaconda python](#). We have used [Enthought Canopy](#) but anaconda seems to work better.

Look on the course site for directions to open Anaconda and then you can move around directories. Remember nothing gets saved on the school computers. so whatever you save each day you need to email to yourself or save some home

Open Jupyter notebook like in the video

Now we are in iPython!!!!

You see that I am using Markdown to add these notes. You make markdown by choosing Markdown above you can change from code to Markdown. Notes are essential to understanding everything. Please make a new cell and add some notes! We will do more of this later. If you download the notebook from Courseworks under syllabus you can double click on the notes and see how I added links! I will show you how to add equations and images later!

But we wanted to analyze precipitation data from Central Park. What questions could we ask and answer? I have downloaded the data from 1876 onward! I want to know the precipitation in the month I was born? I want to know the annual average precipitation in Central Park and I also want to know what monthly Precipitation looks like. Lets see if we can figure that out today. We are going to dive in really deep the first class and show you the power of programming. Then over the course of the semester I will teach you how to do this on your own. So enjoy the first analysis, see what we can do and then work on learning it yourself during the semester.

The first thing we will do is import the python libraries that we will use. When programming you don't use the whole Python programming language. You have a basic part of Python that is always turned-on but also you import or turn on the parts of the language that you will use for your analysis. You will learn about different libraries as the semester progresses.

Type in what you see below. Then to execute the code hit shift-enter. The little circle will fill in on the top right as the computer computes code. It needs to be exact, uppercase, lowercase etc.

TYPE EXACTLY!

If you get a big fat ugly pink error let me know! It is likely.

```
In [1]: %matplotlib ipynb
import pandas as pd
import numpy as np
import matplotlib # for plot data
import matplotlib.pyplot as plt
```

Now we need to get our data. I will use github a lot this semester. It is a website that allows you to update and share code and to keep track of it. It is like Dropbox on steroids. I got the data from NOAA <https://www.ncdc.noaa.gov/cdo-web/datasets/GHCND/stations/GHCND:USW00094728/detail>.

The information for this class is at this website <https://github.com/bmaillou/BigDataPython>

This is the cool part. We don't need to get it. We can just link it below. But you can look at the data and download if you want. follow the link to central park csv file. Then click on it and look at the raw data. csv files are a great way of getting data. csv means commas seperated variables. It is just a text file with data seperated by commas. now copy the link so we can add it below. The link goes to the url. Then we are using the pandas reader to get the data. We are then indexing by Date to organize. In pandas data is usually called "df" for dataframes. It is almost like a monster excel sheet.

```
In [2]: import sys
print(sys.version)

print(pd.show_versions())
```

3.11.13 (main, Jun 5 2025, 08:14:07) [Clang 14.0.6]

INSTALLED VERSIONS

```
-----
commit           : c888af6d0bb674932007623c0867e1fbd4bdc2c6
python           : 3.11.13
python-bits      : 64
OS               : Darwin
OS-release       : 22.6.0
Version          : Darwin Kernel Version 22.6.0: Thu Apr 24 20:25:14 PDT 2025;
root:xnu-8796.141.3.712.2~1/RELEASE_X86_64
machine          : x86_64
processor        : i386
byteorder        : little
LC_ALL           : en_US.UTF-8
LANG             : en_US.UTF-8
LOCALE           : en_US.UTF-8

pandas           : 2.3.1
numpy            : 2.0.1
pytz             : 2025.2
dateutil         : 2.9.0.post0
pip              : 25.2
Cython           : None
sphinx           : None
IPython          : 9.1.0
adbc-driver-postgresql: None
adbc-driver-sqlite   : None
bs4              : 4.13.4
blosc            : None
bottleneck       : 1.4.2
dataframe-api-compat : None
fastparquet      : None
fsspec           : None
html5lib         : None
hypothesis       : None
gcsfs            : None
jinja2           : 3.1.6
lxml.etree       : None
matplotlib       : 3.10.0
numba            : None
numexpr          : 2.11.0
odfpy            : None
openpyxl         : None
pandas_gbq       : None
psycpg2          : None
pymysql          : None
pyarrow          : None
pyreadstat       : None
pytest           : None
python-calamine  : None
pyxlsb           : None
s3fs             : None
scipy            : None
sqlalchemy       : None
tables           : None
tabulate         : None
xarray           : None
xlrd             : None
xlsxwriter       : None
zstandard        : None
```

```

tzdata          : 2025.2
qtpy             : None
pyqt5           : None
None

```

```

In [3]: #watch the \ and / below in order to write one long line over two lines
url='https://raw.githubusercontent.com/bmaillou\
/BigDataPython/master/CentralPark/CentralPark2025.csv'

df = pd.read_csv(url,index_col='DATE',parse_dates=True,date_format='mixed') #This is

```

```
In [ ]:
```

Now the data is in the computer as df. You could just type df to see it. I typed df.head() to see the first 5 rows. Type df.tail() to see the last 5.

```
In [4]: df.tail()
```

```

Out[4]:

```

	STATION	NAME	DASF	MDSF	PRCP	SNOW
DATE						
2025-01-02	USW00094728	NY CITY CENTRAL PARK, NY US	NaN	NaN	0.00	0.0
2025-01-03	USW00094728	NY CITY CENTRAL PARK, NY US	NaN	NaN	0.00	0.0
2025-01-04	USW00094728	NY CITY CENTRAL PARK, NY US	NaN	NaN	0.00	0.0
2025-01-05	USW00094728	NY CITY CENTRAL PARK, NY US	NaN	NaN	0.00	0.0
2025-01-06	USW00094728	NY CITY CENTRAL PARK, NY US	NaN	NaN	0.08	0.9

You can do a lot with the data. For some quick ideas type df. with the period and hit tab. The list of functions comes up.

```
In [ ]:
```

But lets make a quick and dirty plot. To plot all the data you type df.plot() and then to show the plot you type plt.show()

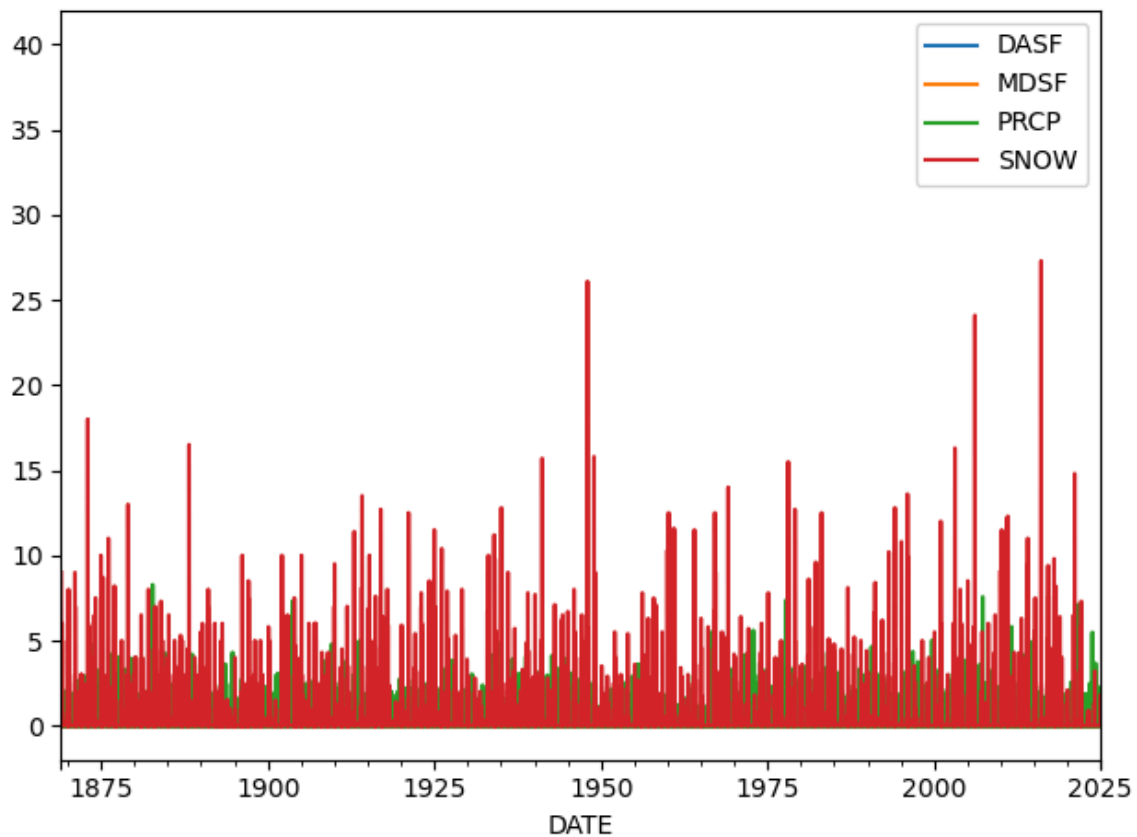
```

In [10]: fig,ax=plt.subplots(layout='tight')
df.plot(ax=ax)

```

```
Out[10]: <Axes: xlabel='DATE'>
```

Figure



I want to know what type of data I have. So i need to check the data type, show how to type data1.
Hit tab and then see what we can do.

```
In [7]: df.dtypes
```

```
Out[7]: STATION    object
NAME          object
DASF          float64
MDSF          float64
PRCP          float64
SNOW          float64
dtype: object
```

But pandas has better built in functions. try df.info() to learn more about your data

```
In [8]: df.info()
```

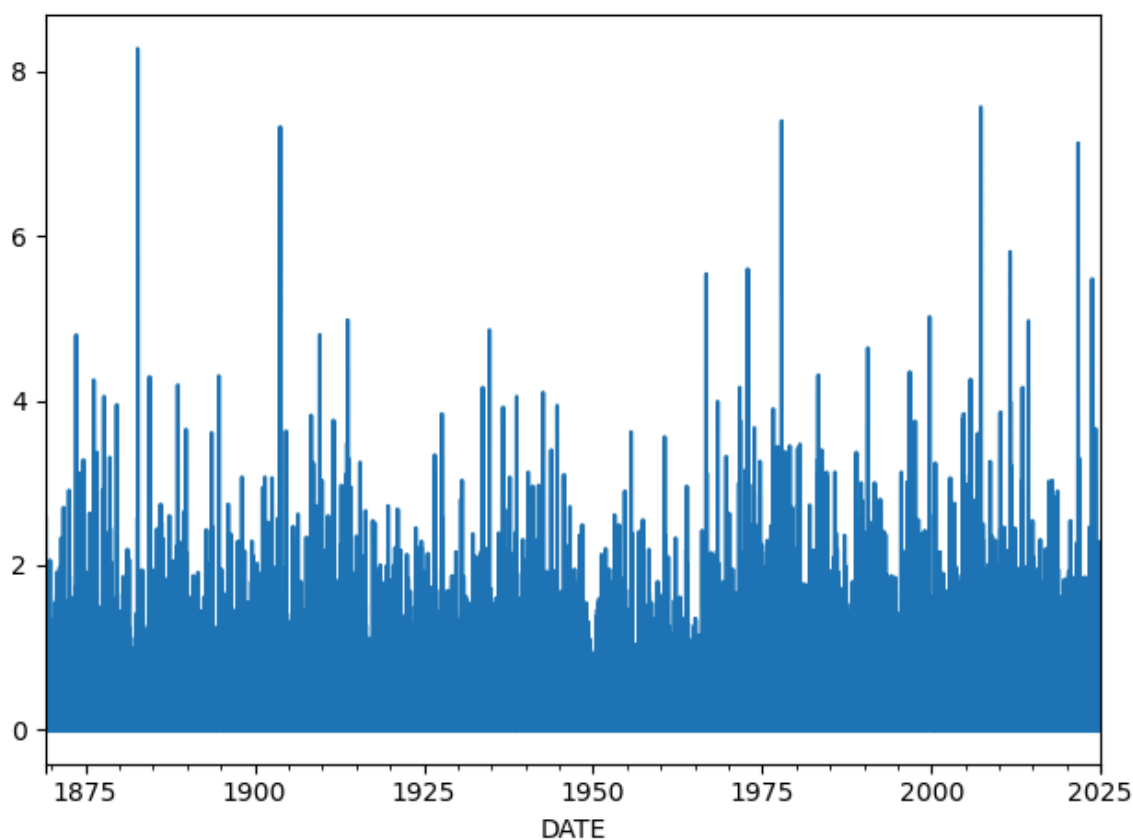
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 56984 entries, 1869-01-01 to 2025-01-06
Data columns (total 6 columns):
#   Column   Non-Null Count  Dtype
---  ---
0    STATION  56984 non-null  object
1    NAME     56984 non-null  object
2    DASF     57 non-null     float64
3    MDSF     57 non-null     float64
4    PRCP     56984 non-null  float64
5    SNOW     56818 non-null  float64
dtypes: float64(4), object(2)
memory usage: 3.0+ MB
```

We are only interested in precipitation today. So lets just plot it. We can subchoose just that column.

```
In [11]: fig,ax=plt.subplots(layout='tight')
         df['PRCP'].plot(ax=ax)
```

```
Out[11]: <Axes: xlabel='DATE'>
```

Figure



We can use describe to look at the description of the data.

```
In [8]: df.describe()
```

Out [8]:

	DASF	MDSF	PRCP	SNOW
count	57.0	57.000000	56984.000000	56818.000000
mean	2.0	5.143333	0.124930	0.073427
std	0.0	5.739186	0.353763	0.622680
min	2.0	0.310000	0.000000	0.000000
25%	2.0	2.010000	0.000000	0.000000
50%	2.0	4.020000	0.000000	0.000000
75%	2.0	6.500000	0.050000	0.000000
max	2.0	40.000000	8.280000	27.300000

Stop and think for a second. What do the results mean above? They say we have over 50,000 precipitation values! We know something about them and we have done this really quickly!

Can you just show the describe for Precipitation? Use what you learned above

In [9]:

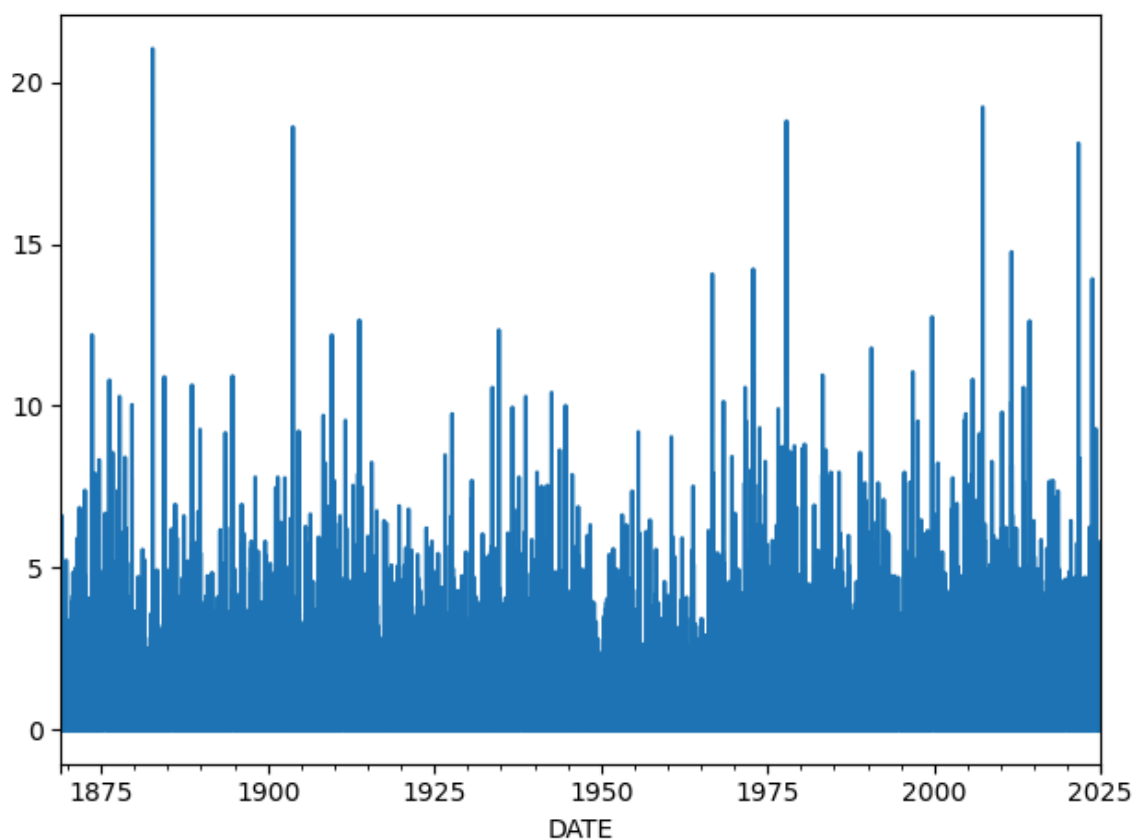
```
Out [9]: count    56984.000000
mean         0.124930
std          0.353763
min          0.000000
25%          0.000000
50%          0.000000
75%          0.050000
max          8.280000
Name: PRCP, dtype: float64
```

We are lucky this year. The data is now in inches! They used to give you the data in 10's of mm. That is cm's but I am not sure why they called it that. So let's convert to cm as practice. 1 in = 2.54 cm.

```
In [12]: fig,ax=plt.subplots(layout='tight')
(df.PRCP*2.54/1).plot(ax=ax) #we are doing the math and plotting at the same time!
```

```
Out [12]: <Axes: xlabel='DATE'>
```

Figure



Again, stop and think about the data? What are the biggest rainstorms we have gotten? How does this compare to some hurricanes you hear about on the news?? or some other big storm near you? What about Ida?

lets convert to cm so we can save it forever!

```
In [37]: df.PRCP=df.PRCP*2.54/1
```

Check and make sure it worked using describe. Does the data make sense?

Make sure not to convert twice above. Everytime you convert is saves it. Your numbers will keep getting smaller and smaller.....

If you convert more than once you have to go up and re-read in the data

```
In [38]:
```

```
Out[38]: count    56064.000000
mean         0.316553
std          0.896502
min          0.000000
25%          0.000000
50%          0.000000
75%          0.127000
max          21.031200
Name: PRCP, dtype: float64
```


I like inches! So lets convert back and then describe again.

```
In [39]: df.PRCP=df.PRCP/2.54
df.PRCP.describe()
```

```
Out[39]: count      56064.000000
mean          0.124627
std           0.352954
min           0.000000
25%           0.000000
50%           0.000000
75%           0.050000
max           8.280000
Name: PRCP, dtype: float64
```

can we just get yearly rainfall? We can resample the data and sum all the rainfall for each year!
Plus look and I am adding a comment to the code below!

```
In [11]: df['PRCP'].resample('A').sum() #This just gave us yearly rainfall!
```

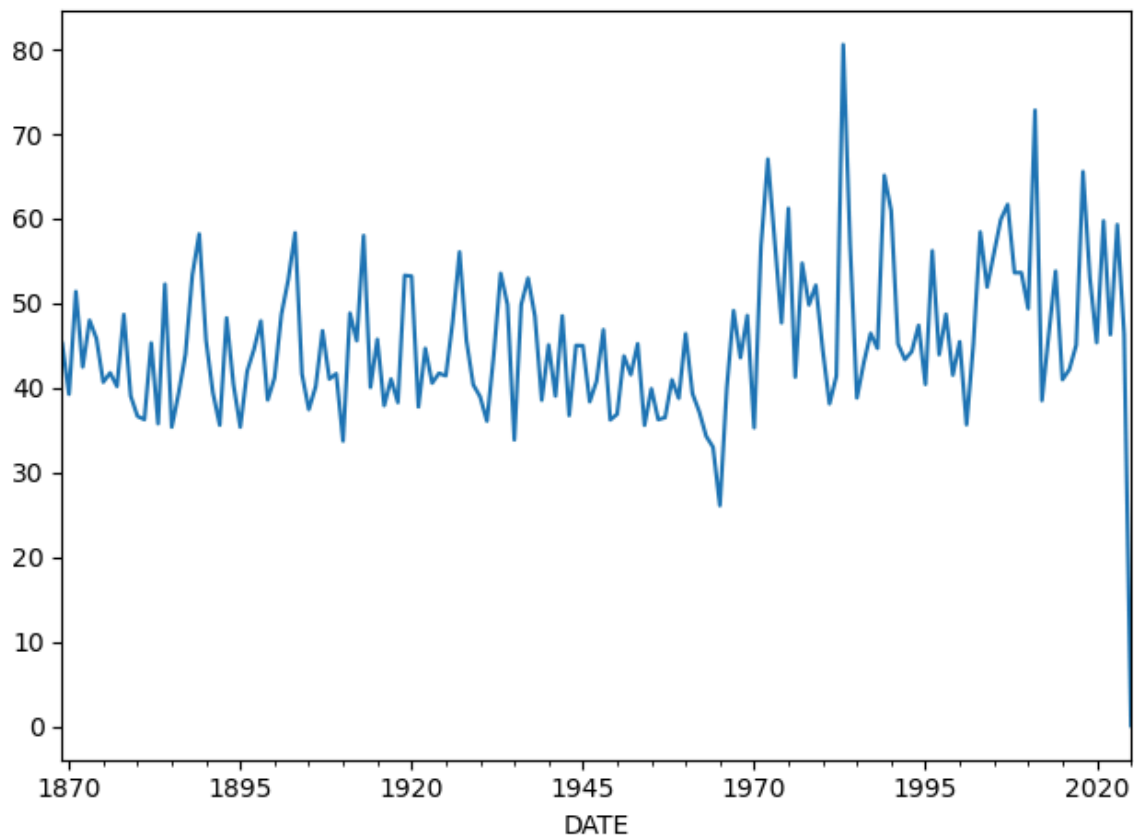
```
Out[11]: DATE
1869-12-31    45.25
1870-12-31    39.25
1871-12-31    51.38
1872-12-31    42.48
1873-12-31    47.99
...
2021-12-31    59.73
2022-12-31    46.30
2023-12-31    59.29
2024-12-31    46.37
2025-12-31     0.08
Freq: A-DEC, Name: PRCP, Length: 157, dtype: float64
```

Now we could plot the yearly data!

```
In [14]: fig,ax=plt.subplots(layout='tight')
df['PRCP'].resample('YE').sum().plot(ax=ax)
```

```
Out[14]: <Axes: xlabel='DATE'>
```

Figure



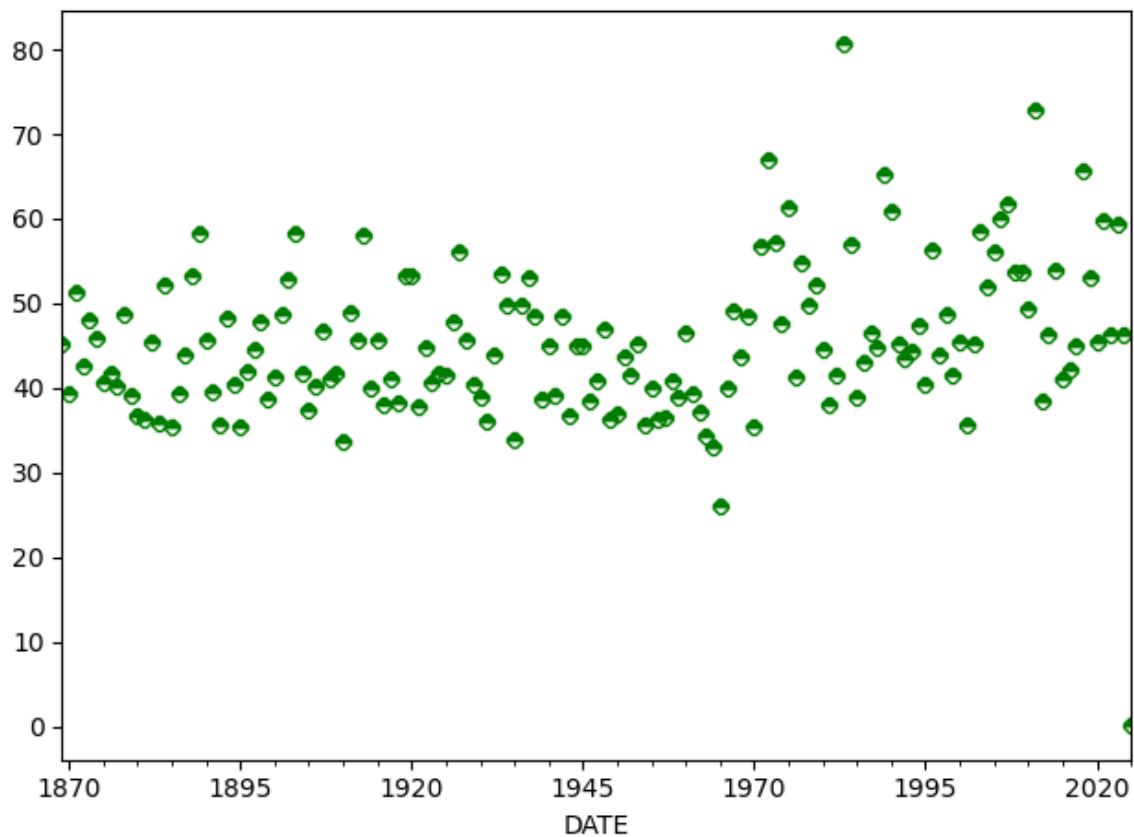
We could also make it a scatter plot with points and no line. This is the power of plotting with matplotlib. You can change anything!

go to http://matplotlib.org/1.3.1/examples/pylab_examples/filledmarker_demo.html and choose your own color and style. if you want to turn the line off do `linestyle='none'`

```
In [16]: fig,ax=plt.subplots(layout='tight')
df['PRCP'].resample('YE').sum().plot(marker='8',color='g'
                                     ,fillstyle='top',linestyle='none'
                                     ,ax=ax)
```

```
Out[16]: <Axes: xlabel='DATE'>
```

Figure



Can you use describe to give statistics on the annual data? How much rain fell on the wettest year?

In [44]:

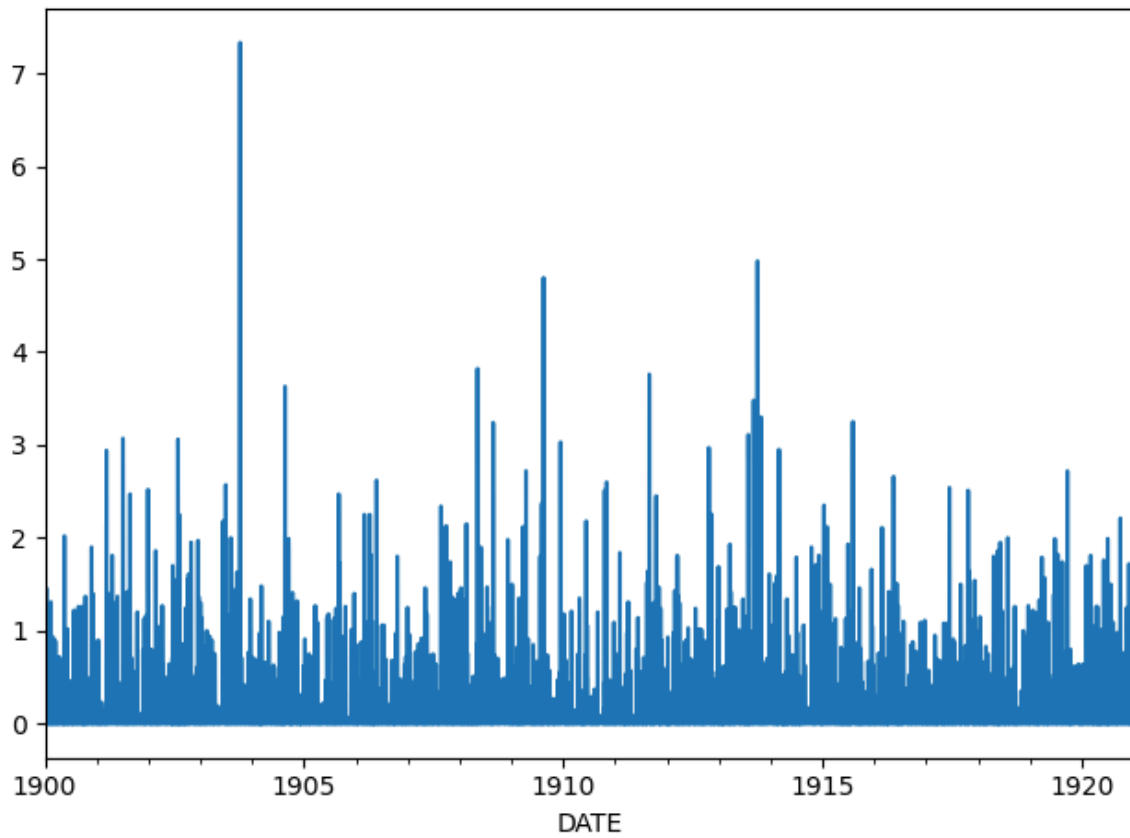
```
Out[44]: count    154.000000
mean      45.370714
std       8.485716
min       26.090000
25%       39.335000
50%       44.415000
75%       49.307500
max       80.560000
Name: PRCP, dtype: float64
```

What if we only want to plot some of the data? Lets plot from 1900 to 1920...

```
In [17]: fig,ax=plt.subplots(layout='tight')
df.PRCP['1900':'1920'].plot(ax=ax)
```

```
Out[17]: <Axes: xlabel='DATE'>
```

Figure

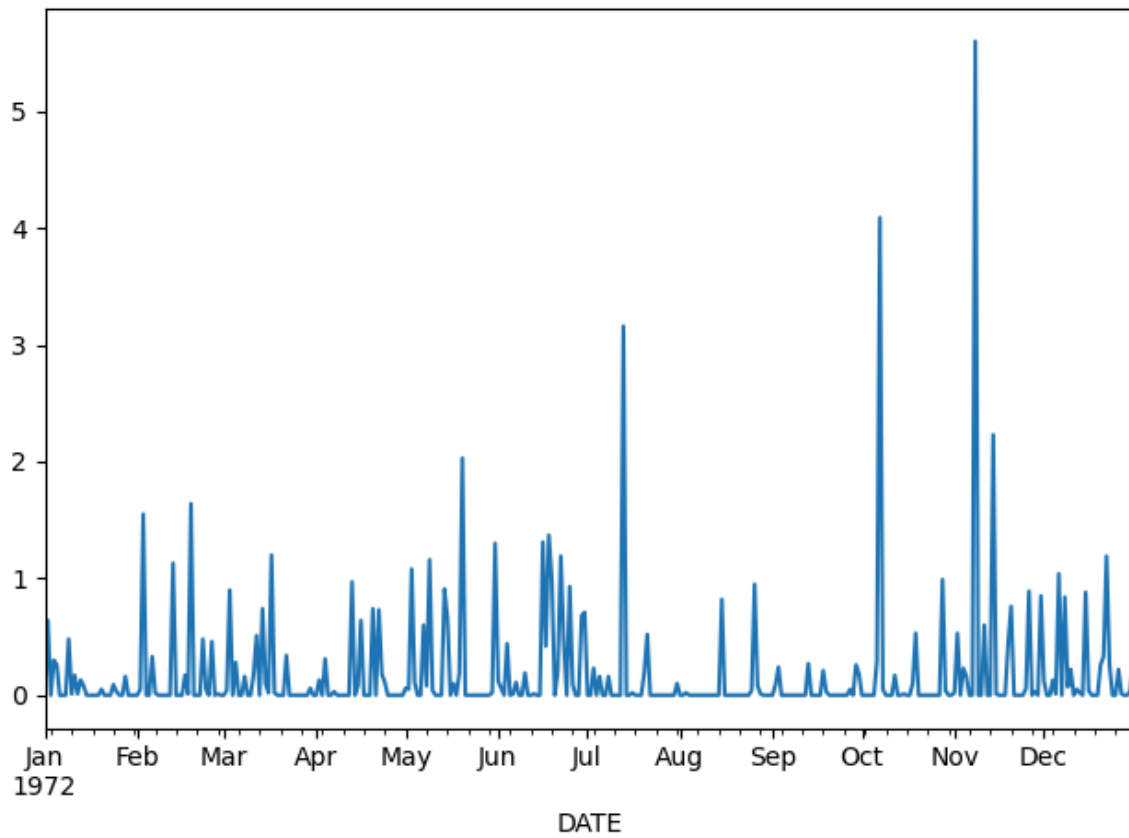


or for just one year

```
In [18]: fig,ax=plt.subplots(layout='tight')
         df.PRCP['1972'].plot(ax=ax)
```

```
Out[18]: <Axes: xlabel='DATE'>
```

Figure

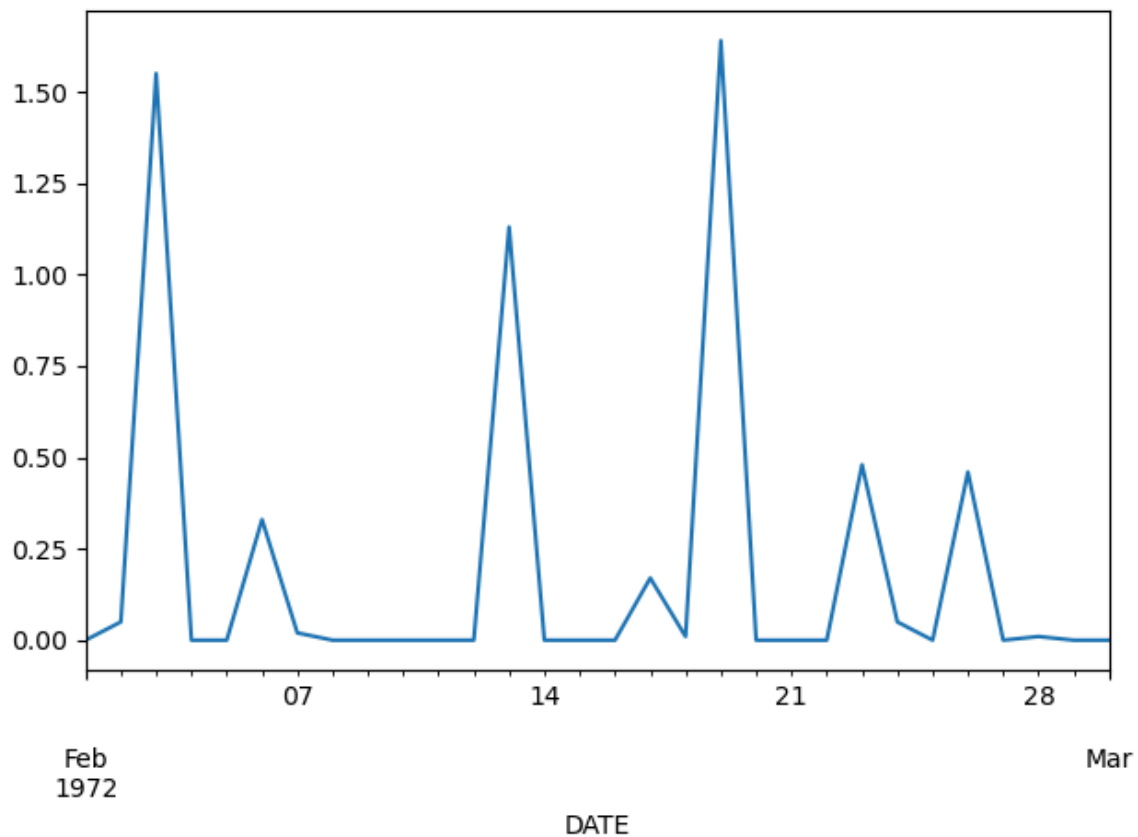


or one month!

```
In [19]: fig,ax=plt.subplots(layout='tight')
df.PRCP['02/01/1972':'03/01/1972'].plot(ax=ax)
```

```
Out[19]: <Axes: xlabel='DATE'>
```

Figure



Can you plot for the month and or year you were born?

In []:

Now we are going to do some big changes. (I am also showing you here that you can do bullets in the Markdown)

My end point is to look at the monthly data in boxplots.

- But to get there we need to resample to monthly.
- Then we need to make a new column that lists the months.
- Then we can make a boxplot!

So lets start by making a new dataframe which is the data resampled by summing the monthly data.

```
In [32]: df_month=df.PRCP.resample('ME').sum().to_frame()
```

Now we can describe the data

```
In [33]: df_month.describe()
```

Out [33]:

PRCP	
count	1873.000000
mean	3.800854
std	2.141921
min	0.010000
25%	2.310000
50%	3.440000
75%	4.820000
max	18.950000

So things appear to be working. What does our data look like?

In [34]: `df_month`

Out [34]:

PRCP	
DATE	
1869-01-31	2.53
1869-02-28	6.87
1869-03-31	4.61
1869-04-30	1.39
1869-05-31	4.15
...	...
2024-09-30	1.58
2024-10-31	0.01
2024-11-30	3.35
2024-12-31	4.53
2025-01-31	0.08

1873 rows × 1 columns

To make our plotting work we will need a new column that tells us just the month. So we are going to look at the index date and pull out the month and put it into a new column.

In [35]: `df_month['month']=df_month.index.month`

lets just look at 5 rows and see if it worked.

In [36]: `df_month.head()` *#head just gives you the first 5*

Out[36]:

	PRCP	month
DATE		
1869-01-31	2.53	1
1869-02-28	6.87	2
1869-03-31	4.61	3
1869-04-30	1.39	4
1869-05-31	4.15	5

Now lets make our boxplot! you can google pandas boxplot to see how.

<http://pandas.pydata.org/pandas-docs/stable/visualization.html>

Foreshadowing. I am going to ask you to use this by notation in a future class!

this is a big hint. remember column and by. I AM GOING TO ASK THIS AGAIN!

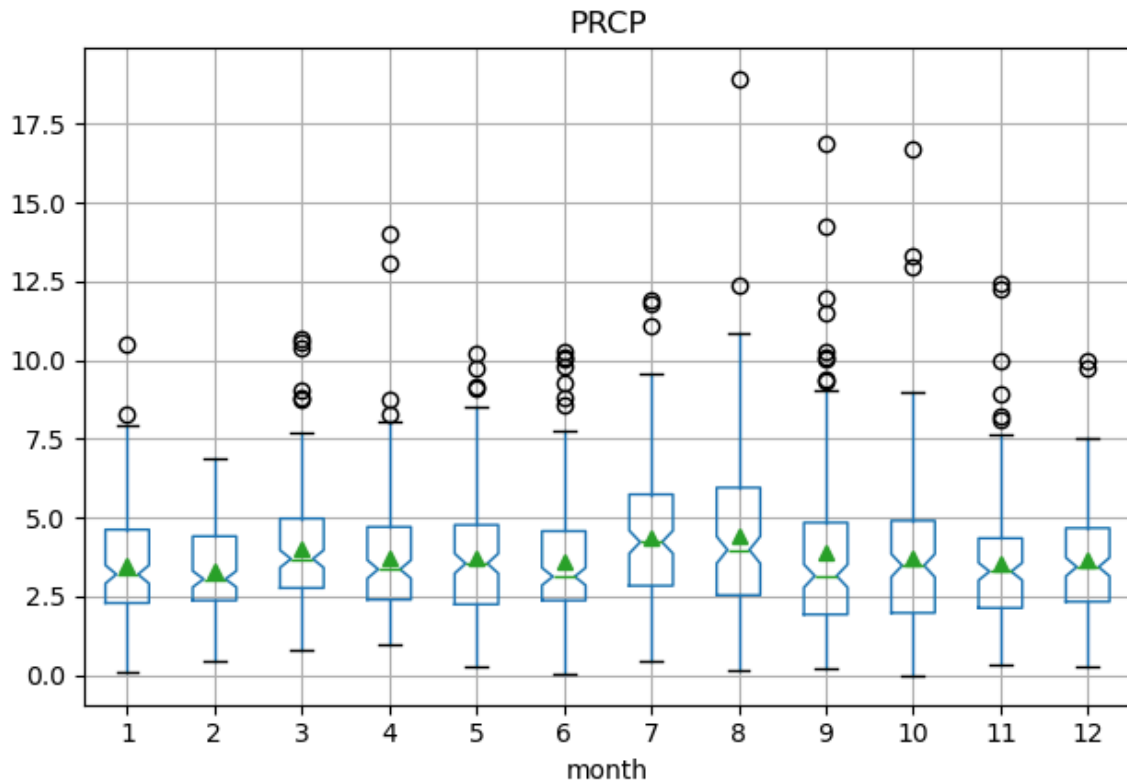
Also, to make it look fancy I am showing the mean values and making it have a notch. You can make both options False or just delete them. They are extra arguments in the function.

```
In [37]: fig,ax=plt.subplots(layout='tight')
df_month.boxplot(column=['PRCP'],by='month',
                  ,showmeans=True,notch=True
                  ,ax=ax)
```

```
Out[37]: <Axes: title={'center': 'PRCP'}, xlabel='month'>
```


Figure

Boxplot grouped by month



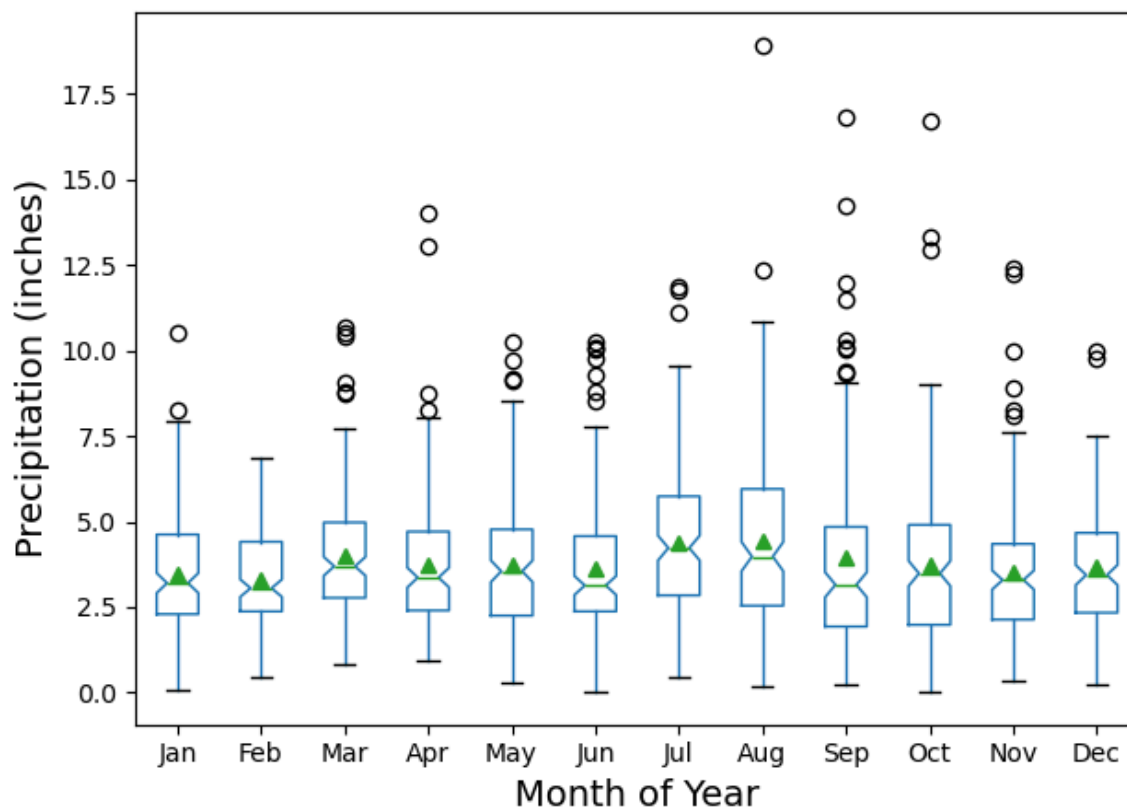
the great thing about python is you can make a professional looking graph and save it and have it ready to use. So lets do it.

- First we remove the title and suptitle by making them nothing. `ax.set_title('')` `fig.suptitle('')`
You could add titles if you wanted.
- We label our y-axis. Always label axes. `ax.set_ylabel('Precipitation (inches)')`
- We make the x-axis nicer.
- We make the label fonts bigger
- We save the file. I added some keywords to make the file nice. the dpi is dots per inch so it has better resolution and the bbox makes sure nothing is cut off.
- You can decide if you want to turn off gridlines. add `grid=False`
- for a hard one we could add month of year instead of 1-12. I used
`ax.set_xticklabels(['Jan','Feb','Mar','Apr','May','Jun','Jul','Aug','Sep','Oct','Nov','Dec'])`

I think that is a sharp graph! You should be able to find it on your computer and mail it to yourself.

```
In [40]: fig,ax=plt.subplots(layout='tight')
df_month.boxplot(column=['PRCP'],by='month',
                  ,showmeans=True,notch=True,ax=ax,grid=False)
ax.set_title('')
fig.suptitle('')
ax.set_ylabel('Precipitation (inches)',fontsize=14)
ax.set_xlabel('Month of Year',fontsize=14)
ax.set_xticklabels(['Jan','Feb','Mar','Apr','May','Jun',
                  , 'Jul','Aug','Sep','Oct','Nov','Dec'])
fig.savefig('Central-Park-Monthly-Precip.jpg',dpi=600,bbox_inches='tight')
```

Figure



I talk about making figure captions in the homework. So lets make one here. Here is the definition of a [boxplot](#)

Figure 1. Boxplot of monthly precipitation Central Park, New York City from 1876 to 2015. The indent is the median, the triangle is the mean, the box goes from Q1 to Q3 with the whiskers showing the range of the data and circles are outliers larger than 1.5 times the interquartile range.

We will also need to learn about how to talk about results. We can see from the figure that precipitation does not systematically vary by month in Central Park and that the mean monthly precipitation is fairly uniform.

to get the monthly data we can groupby month and the describe the data. or just look at the mean. We will also only look at prcp. As you can see rain is pretty uniform by month in NYC!

```
In [38]: df_month.groupby('month').PRCP.mean()
```

```
Out[38]: month
1      3.479299
2      3.323718
3      3.990449
4      3.744231
5      3.728397
6      3.633718
7      4.393333
8      4.450128
9      3.925064
10     3.717692
11     3.534231
12     3.692051
Name: PRCP, dtype: float64
```

I hope you got some of these to work and are excited about what we can do. You just analyzed 50,000 precipitation data points on your first day! We are ready to begin learning data analysis and Python! But for next class we have to go back to the basics!

```
In [ ]:
```